Boxuan Yu 22.12.2025

Mini Project – Fall Detection for Elderly Support

1. Project Goal: To design a fall detection system for old people in daily life.
2. Sensor Selection: Inertial Measurement Unit (IMU) on the PBL Shield.
3. Algorithms:
    a. Acceleration Vector Magnitude: This algorithm provides an orientation-independent measure of the total Gravitational force. With the sensor it can detect an instantaneous increase in acceleration.
    b. Tilt Angle Calculation: This algorithm uses trigonometry to determine the board's inclination according to the vertical axis. People are allowed to change threshold to fix the best detected angle.
4. Performance Profiling: The profiling is conducted by using the DWT cycle counter to measure computational efficiency:
    a. Acceleration Vector Magnitude: ca. 134 clock cycles.
    b. Tilt Angle Calculation: ca. 460 clock cycles.

    The second algorithm (Angle Calculation) needs more clock cycles because of the trigonometric calculation.

5. Energy Consumption Evaluation:
    $System\ Frequency\ =\ 80\ MHz$

    $Active\ Power = 0.5\ {}^{mW}/_{MHz} \cdot 80\ MHz = 40\ mW$

    $Sleep\ Power = 15\ \mu W$

    a. Active Mode:
        - Acceleration Task Energy: E $\approx 40\ mW\ \cdot\ {}^{134}/_{80\ MHz} = 0.067\ \mu J$
        - Angle Calculation Energy: $E \approx 40\ mW\ \cdot\ {}^{460}/_{80\ MHz} = 0.23\ \mu J$
    b. Sleep Mode: $E\ =\ 15\ \mu W\ \cdot\ 5\ s\ =\ 75\ \mu J$

    So the total Energy per Period is $75.3\ \mu J$.

    Comparison: The energy consume of the sleep mode is smaller than the active mode, and the sleep mode also dominates each period. Therefore, it can save more energy.

6. Code Excerpts:
   a. Acceleration Vector Magnitude:

```
float Get_Vector(void) {
    uint8_t raw[6];
    IMU_Read(reg: OUTX_L_XL, data: raw, len: 6);
    int16_t x = (int16_t)(raw[1] << 8 | raw[0]);
    int16_t y = (int16_t)(raw[3] << 8 | raw[2]);
    int16_t z = (int16_t)(raw[5] << 8 | raw[4]);
    float xg = x * SENSITIVITY_4G;
    float yg = y * SENSITIVITY_4G;
    float zg = z * SENSITIVITY_4G;
    return sqrtf(xg*xg + yg*yg + zg*zg);
}
```

   b. Tilt Angle Calculation:

```
float Get_Angle(void) {
    uint8_t raw[6];
    // Read acceleration data (SPI read 6 bytes)
    IMU_Read(reg: OUTX_L_XL, data: raw, len: 6);

    // Convert raw data
    // angle calculation temporarily not using X axis
    int16_t y = (int16_t)(raw[3] << 8 | raw[2]);
    int16_t z = (int16_t)(raw[5] << 8 | raw[4]);

    // Convert to physical units (g)
    float yg = y * SENSITIVITY_4G;
    float zg = z * SENSITIVITY_4G;

    // Calculate angle (using atan2)
    float angle_rad = atan2f(yg, zg);
    float angle_deg = angle_rad * 57.296f; // 57.296f = (180 / PI)

    return fabsf(angle_deg); // Take absolute value
}
```